

Cloud Computing: Applications and Paradigms

The efforts to support large-scale distributed computing have encountered major difficulties over the years. The users of these systems discovered how difficult it was to locate the systems able to run an application. They soon realized that it is equally difficult to scale up and down to accommodate a dynamic load, to recover after a system failure, and to efficiently support checkpoint or restarting procedures.

At the same time, the providers of computing cycles and storage realized the difficulties in managing a large number of systems and providing guarantees for the quality of service. Any economic advantage offered by resource concentration was offset by the cost of management and the relatively low utilization of costly resources.

Cloud computing is very attractive to users for several economic reasons: It requires a very low infrastructure investment because there is no need to assemble and maintain a large-scale system and it has low *utility-based computing costs* because customers are only billed for the infrastructure used. At the same time, users benefit from the potential to reduce the execution time of compute-intensive and data-intensive applications through parallelization. If an application can partition the workload in n segments and spawn n instances of itself, the execution time could be reduced by a factor close to n .

Moreover, because application developers enjoy the advantages of a *just-in-time infrastructure*, they are free to design an application without being concerned with the system where the application will run. Often, an application becomes the victim of its own success, attracting a user population larger than the system can support. Cloud elasticity allows such an application to absorb the additional workload without any effort from application developers.

Cloud computing is also beneficial for the providers of computing cycles because it typically leads to more efficient resource utilization. The future success of cloud computing rests on the ability of companies promoting utility computing to convince an increasingly large segment of the user population of the advantages of network-centric computing and content. This translates into the ability to provide satisfactory solutions to critical aspects of security, scalability, reliability, quality of service, and the requirements enforced by SLAs.

The appeal of cloud computing is its focus on enterprise applications. This clearly differentiates it from the grid computing effort, which was largely focused on scientific and engineering applications. Of course, the other major advantage of the cloud computing approach over grid computing is the concentration of resources in large data centers in a single administrative domain.

It is expected that utility computing providers such as Amazon, Apple, Google, HP, IBM, Microsoft, Oracle, and others will in the future develop application suites to attract customers. Microsoft seems well positioned in the effort to attract customers to its Azure platform since it already has many applications for enterprise customers, while Red Hat and Amazon may choose to stay with their “infrastructure only” approach [27].

The main attraction of cloud computing is the ability to use as many servers as necessary to optimally respond to the cost and the timing constraints of an application. This is possible only if the workload can be partitioned in segments of arbitrary size and can be processed in parallel by the servers available in the cloud. In Section 4.6 we discuss *arbitrarily divisible* workloads that can be partitioned into an arbitrarily large number of segments; the *arbitrarily divisible load-sharing model* is common to many applications, and these are precisely the applications suitable for cloud computing.

Web services, database services, and transaction-based services are ideal applications for cloud computing. The resource requirements of transaction-oriented services are dependent on the current load, which itself is very dynamic; the cost/performance profile of such applications benefits from an elastic environment in which resources are available when needed and users pay only for the resources they consume.

Not all applications are suitable for cloud computing; applications for which the workload cannot be arbitrarily partitioned or that require intensive communication among concurrent instances are unlikely to perform well on a cloud. An application with a complex workflow and multiple dependencies, as is often the case in high-performance computing, could require longer execution times and higher costs on a cloud. The benchmarks for high-performance computing discussed in Section 4.9 show that communication and memory-intensive applications may not exhibit the performance levels shown when running on supercomputers with low latency and high-bandwidth interconnects.

4.1 Challenges for cloud computing

The development of efficient cloud applications inherits the challenges posed by the natural imbalance among computing, I/O, and communication bandwidths of physical systems. These challenges are greatly amplified due to the scale of the system, its distributed nature, and the fact that virtually all applications are data-intensive. Though cloud computing infrastructures attempt to automatically distribute and balance a load, the application developer is still left with the responsibility of placing the data close to the processing site and identifying optimal storage for the data.

One of the main advantages of cloud computing, the shared infrastructure, could also have a negative impact. Performance isolation¹ is nearly impossible to reach in a real system, especially when the system is heavily loaded. The performance of virtual machines fluctuates based on the load, the infrastructure services, and the environment, including the other users. Security isolation is also challenging on multi-tenant systems.

Reliability is also a major concern; node failures are to be expected whenever a large number of nodes cooperate for the computations. Choosing an optimal instance (in terms of performance isolation, reliability, and security) from those offered by the cloud infrastructure is another critical factor to be considered. Of course, cost considerations also play a role in the choice of the instance type.

Many applications consist of multiple stages; in turn, each stage may involve multiple instances running in parallel on the systems of the cloud and communicating among them. Thus, efficiency, consistency, and communication scalability are major concerns for an application developer. Indeed,

¹Performance and security isolation of virtual machines are discussed in Section 5.5.

due to shared networks and unknown topology, cloud infrastructures exhibit internode latency and bandwidth fluctuations that affect application performance.

Data storage plays a critical role in the performance of any data-intensive application; the organization of the storage, the storage location, and the storage bandwidth must be carefully analyzed to lead to optimal application performance. Clouds support many storage options to set up a file system similar to the *Hadoop* file system discussed in Section 8.6; among them are off-instance cloud storage (e.g., S3), mountable off-instance block storage (e.g., *EBS*), and storage persistent for the lifetime of the instance.

Many data-intensive applications use metadata associated with individual data records; for example, the metadata for an MPEG audio file may include the name of the song, the singer, recording information, and so on. Metadata should be stored for easy access, and the storage should be scalable and reliable.

Another important consideration for the application developer is logging. Performance considerations limit the amount of data logging, whereas the ability to identify the source of unexpected results and errors is helped by frequent logging. Logging is typically done using instance storage preserved only for the lifetime of the instance. Thus, measures to preserve the logs for a postmortem analysis must be taken. Another challenge awaiting resolution is related to software licensing, discussed in Section 3.11.

4.2 Existing cloud applications and new application opportunities

Existing cloud applications can be divided into several broad categories: (i) processing pipelines; (ii) batch processing systems; and (iii) Web applications [360].

Processing pipelines are data-intensive and sometimes compute-intensive applications and represent a fairly large segment of applications currently running on the cloud. Several types of data processing applications can be identified:

- *Indexing*. The processing pipeline supports indexing of large datasets created by Web crawler engines.
- *Data mining*. The processing pipeline supports searching very large collections of records to locate items of interests.
- *Image processing*. A number of companies allow users to store their images on the cloud (e.g., *Flickr* (www.flickr.com) and *Google* (<http://picasa.google.com/>)). The image-processing pipelines support image conversion (e.g., enlarging an image or creating thumbnails). They can also be used to compress or encrypt images.
- *Video transcoding*. The processing pipeline transcodes from one video format to another (e.g., from AVI to MPEG).
- *Document processing*. The processing pipeline converts very large collections of documents from one format to another (e.g., from *Word* to *PDF*), or encrypts the documents. It could also use optical character recognition (OCR) to produce digital images of documents.

Batch processing systems also cover a broad spectrum of data-intensive applications in enterprise computing. Such applications typically have deadlines, and the failure to meet these deadlines could

have serious economic consequences. Security is also a critical aspect for many applications of batch processing. A nonexhaustive list of batch processing applications includes:

- Generation of daily, weekly, monthly, and annual activity reports for organizations in retail, manufacturing, and other economic sectors.
- Processing, aggregation, and summaries of daily transactions for financial institutions, insurance companies, and healthcare organizations.
- Inventory management for large corporations.
- Processing billing and payroll records.
- Management of the software development (e.g., nightly updates of software repositories).
- Automatic testing and verification of software and hardware systems.

Finally, and of increasing importance, are cloud applications in the area of Web access. Several categories of Web sites have a periodic or a temporary presence, such as the Web sites for conferences or other events. There are also Web sites that are active during a particular season (e.g., the holiday season) or that support a particular type of activity, such as income tax reporting with the April 15 deadline each year. Other limited-time Web sites used for promotional activities “sleep” during the night and auto-scale during the day.

It makes economic sense to store the data in the cloud close to where the application runs; as we saw in Section 3.1, the cost per GB is low and the processing is much more efficient when the data is stored close to the computational servers. This leads us to believe that several new classes of cloud computing applications could emerge in the years to come – for example, batch processing for decision support systems and other aspects of business analytics. Another class of new applications could be parallel batch processing based on programming abstractions, such as *MapReduce*, discussed in Section 4.6. Mobile interactive applications that process large volumes of data from different types of sensors and services that combine more than one data source (e.g., mashups²) are obvious candidates for cloud computing.

Science and engineering could greatly benefit from cloud computing because many applications in these areas are compute- and data-intensive. Similarly, a cloud dedicated to education would be extremely useful. Mathematical software such as *MATLAB* and *Mathematica* could also run on the cloud.

4.3 Architectural styles for cloud applications

Cloud computing is based on the client-server paradigm discussed in Section 2.13. The vast majority of cloud applications take advantage of request/response communication between clients and stateless servers. A *stateless server* does not require a client to first establish a connection to the server. Instead, it views a client request as an independent transaction and responds to it.

The advantages of stateless servers are obvious. Recovering from a server failure requires considerable overhead for a server that maintains the state of all its connections, whereas in the case of a

²A *mashup* is an application that uses and combines data, presentation, or functionality from two or more sources to create a service. The fast integration, frequently using open APIs and multiple data sources, produces results not envisioned by the original services. Combination, visualization, and aggregation are the main attributes of mashups.

stateless server a client is not affected while a server goes down and then comes back up between two consecutive requests. A stateless system is simpler, more robust, and scalable. A client does not have to be concerned with the state of the server. If the client receives a response to a request, that means that the server is up and running; if not, it should resend the request later. A connection-based service must reserve spaces to maintain the state of each connection with a client; therefore, such a system is not scalable, and the number of clients a server could interact with at any given time is limited by the storage space available to the server.

For example, a basic Web server is stateless; it responds to an HTTP request without maintaining a history of past interactions with the client. The client, a browser, is also stateless since it sends requests and waits for responses. The *Hypertext Transfer Protocol (HTTP)* used by a browser to communicate with the Web server is a request/response application protocol. HTTP uses the *Transport Control Protocol (TCP)*, a connection-oriented and reliable transport protocol. The use of TCP ensures reliable delivery of large objects but exposes the Web servers to denial-of-service attacks when malicious clients fake attempts to establish a TCP connection and force the server to allocate space for the connection.

A critical aspect of the development of networked applications is how processes and threads running on systems with different architectures and possibly compiled from different programming languages can *communicate structured information with one another*. First, the internal representation of the two structures at the two sites may be different. One system may use *Big-Endian* and the other *Little-Endian* representation. The character representations may also be different. Second, a communication channel transmits a sequence of bits and bytes; thus, the data structure must be serialized at the sending site and reconstructed at the receiving site.

Several other considerations must be analyzed before deciding on the architectural style of an application. The term *neutrality* refers to the ability of the application protocol to use different transport protocols such as TCP or UDP and, in general, to run on top of a different protocol stack. For example, we shall see that SOAP can use TCP but also UDP, SMTP,³ or JMS⁴ as transport vehicles. *Extensibility* refers to the ability to incorporate additional functions, such as security. *Independence* refers to the ability to accommodate different programming styles.

Very often the application clients and the servers running on the cloud communicate using RPCs, discussed in Section 2.13, but other styles of communication are possible. RPC-based applications use *stubs* to convert the parameters involved in an RPC call. A stub performs two functions: marshalling the data structures and serialization. A more general concept is that of an *Object Request Broker (ORB)*, the middleware that facilitates communication of networked applications. The ORB at the sending site transforms the data structures used internally by a sending process to a byte sequence and transmits this byte sequence over the network. The ORB at the receiving site maps the byte sequence to the data structures used internally by the receiving process.

The *Common Object Request Broker Architecture (CORBA)* was developed in the early 1990s to allow networked applications developed in different programming languages and running on systems with different architectures and system software to work with one another. At the heart of the system is the *Interface Definition Language (IDL)*, used to specify the interface of an object. The IDL representation

³Simple Mail Transfer Protocol (SMTP) is an application protocol defined in the early 1980s to support email services.

⁴Java Message Service (JMS) is a middleware of the Java Platform for sending messages between two or more clients.

is then mapped to the set of programming languages, including *C*, *C++*, *Java*, *Smalltalk*, *Ruby*, *LISP*, and *Python*. Networked applications pass *CORBA* by reference and pass data by value.

The *Simple Object Access Protocol* (*SOAP*) is an application protocol developed in 1998 for Web applications; its message format is based on the *Extensible Markup Language* (*XML*). *SOAP* uses *TCP* and, more recently, *UDP* transport protocols. It can also be stacked above other application layer protocols such as *HTTP*, *SMTP*, or *JMS*. The processing model of *SOAP* is based on a network consisting of senders, receivers, intermediaries, message originators, ultimate receivers, and message paths. *SOAP* is an underlying layer of Web Services.

The *Web Services Description Language* (*WSDL*) (see www.w3.org/TR/wsdl) was introduced in 2001 as an *XML*-based grammar to describe communication between endpoints of a networked application. The abstract definition of the elements involved include *services*, collections of endpoints of communication; *types*, containers for data type definitions; *operations*, descriptions of actions supported by a service; *port types*, operations supported by endpoints; *bindings*, protocols and data formats supported by a particular port type; and *port*, an endpoint as a combination of a binding and a network address. These abstractions are mapped to concrete message formats and network protocols to define endpoints and services.

Representational State Transfer (*REST*) is a style of software architecture for distributed hypermedia systems. *REST* supports client communication with stateless servers. It is platform- and language-independent, supports data caching, and can be used in the presence of firewalls.

REST almost always uses *HTTP* to support all four *Create/Read/Update/Delete* (*CRUD*) operations. It uses *GET*, *PUT*, and *DELETE* to read, write, and delete the data, respectively. *REST* is a much easier-to-use alternative to *RPC*, *CORBA*, or Web Services such as *SOAP* or *WSDL*. For example, to retrieve the address of an individual from a database, a *REST* system sends a URL specifying the network address of the database, the name of the individual, and the specific attribute in the record the client application wants to retrieve – in this case, the address. The corresponding *SOAP* version of such a request consists of 10 lines or more of *XML*. The *REST* server responds with the address of the individual. This justifies the statement that *REST* is a lightweight protocol. As far as usability is concerned, *REST* is easier to build from scratch and to debug, but *SOAP* is supported by tools that use self-documentation (e.g., *WSDL* to generate the code to connect).

4.4 Workflows: Coordination of multiple activities

Many cloud applications require the completion of multiple interdependent tasks; the description of a complex activity involving such an ensemble of tasks is known as a *workflow*. In this section we discuss workflow models, the life cycle of a workflow, the desirable properties of a workflow description, workflow patterns, reachability of the goal state of a workflow, and dynamic workflows and conclude with a parallel between traditional transaction systems and cloud workflows [230].

Workflow models are abstractions revealing the most important properties of the entities participating in a workflow management system. *Task* is the central concept in workflow modeling; a task is a unit of work to be performed on the cloud, and it is characterized by several attributes, such as:

- *Name*. A string of characters uniquely identifying the task.
- *Description*. A natural language description of the task.

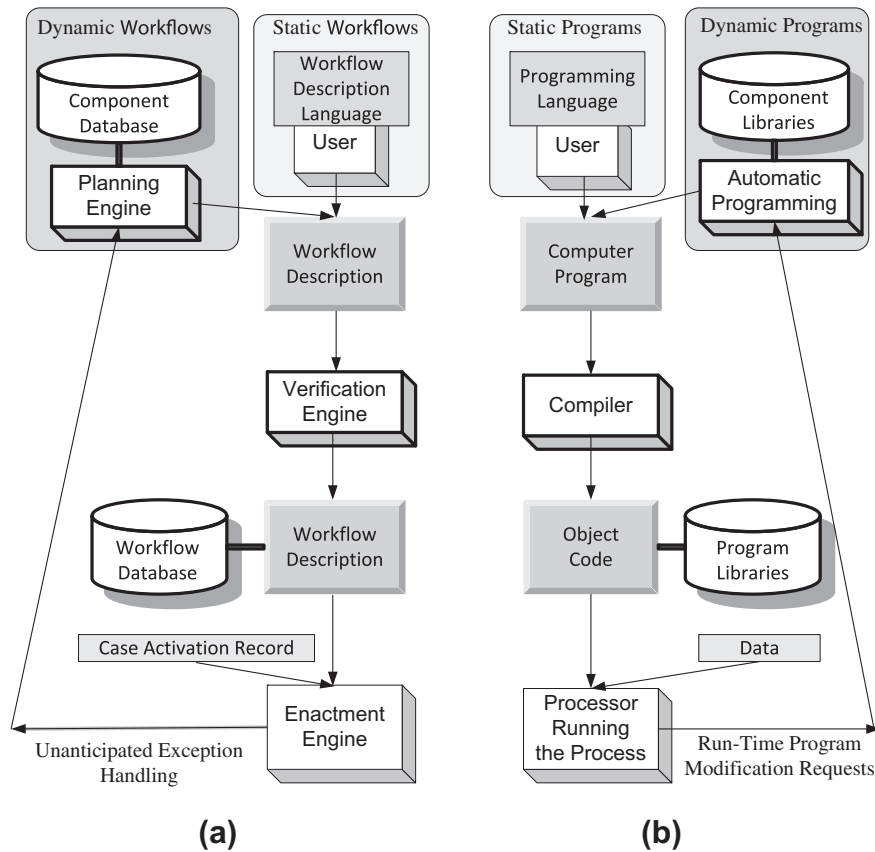
- *Actions*. Modifications of the environment caused by the execution of the task.
- *Preconditions*. Boolean expressions that must be true before the action(s) of the task can take place.
- *Post-conditions*. Boolean expressions that must be true after the action(s) of the task take place.
- *Attributes*. Provide indications of the type and quantity of resources necessary for the execution of the task, the actors in charge of the tasks, the security requirements, whether the task is reversible, and other task characteristics.
- *Exceptions*. Provide information on how to handle abnormal events. The exceptions supported by a task consist of a list of `<event, action>` pairs. The exceptions included in the task exception list are called *anticipated exceptions*, as opposed to unanticipated exceptions. Events not included in the exception list trigger replanning. *Replanning* means restructuring of a process or redefinition of the relationship among various tasks.

A *composite task* is a structure describing a subset of tasks and the order of their execution. A *primitive task* is one that cannot be decomposed into simpler tasks. A composite task inherits some properties from workflows; it consists of tasks and has one start symbol and possibly several end symbols. At the same time, a composite task inherits some properties from tasks; it has a name, preconditions, and post-conditions.

A *routing task* is a special-purpose task connecting two tasks in a workflow description. The task that has just completed execution is called the *predecessor* task; the one to be initiated next is called the *successor task*. A routing task could trigger a sequential, concurrent, or iterative execution. Several types of routing task exist:

- A *fork routing task* triggers execution of several successor tasks. Several semantics for this construct are possible:
 - All successor tasks are enabled.
 - Each successor task is associated with a condition. The conditions for all tasks are evaluated, and only the tasks with a `true` condition are enabled.
 - Each successor task is associated with a condition. The conditions for all tasks are evaluated, but the conditions are mutually exclusive and only one condition may be `true`. Thus, only one task is enabled.
 - Nondeterministic, k out of $n > k$ successors are selected at random to be enabled.
- A *join routing task* waits for completion of its predecessor tasks. There are several semantics for the join routing task:
 - The successor is enabled after all predecessors end.
 - The successor is enabled after k out of $n > k$ predecessors end.
 - Iterative: The tasks between the fork and the join are executed repeatedly.

A *process description*, also called a *workflow schema*, is a structure describing the *tasks* or *activities* to be executed and the order of their execution. A process description contains one start symbol and one end symbol. A process description can be provided in a *workflow definition language (WFDL)*, supporting constructs for choice, concurrent execution, the classical *fork*, *join* constructs, and iterative execution. Clearly, a workflow description resembles a *flowchart*, a concept we are familiar with from programming.

**FIGURE 4.1**

A parallel between workflows and programs. (a) The life cycle of a workflow. (b) The life cycle of a computer program. The workflow definition is analogous to writing a program. Planning is analogous to automatic program generation. Verification corresponds to syntactic verification of a program. Workflow enactment mirrors the execution of a program. A static workflow corresponds to a static program and a dynamic workflow to a dynamic program.

The phases in the life cycle of a workflow are creation, definition, verification, and enactment. There is a striking similarity between the life cycle of a workflow and that of a traditional computer program, namely, creation, compilation, and execution (see Figure 4.1). The workflow specification by means of a workflow description language is analogous to writing a program. Planning is equivalent to automatic program generation. Workflow verification corresponds to syntactic verification of a program, and workflow enactment mirrors the execution of a compiled program.

A *case* is an instance of a process description. The start and stop symbols in the workflow description enable the creation and the termination of a case, respectively. An *enactment model* describes the steps

taken to process a case. When a computer executes all tasks required by a workflow the enactment can be performed by a program called an *enactment engine*.

The *state of a case* at time t is defined in terms of tasks already completed at that time. Events cause transitions between states. Identifying the states of a case consisting of concurrent activities is considerably more difficult than identifying the states of a strictly sequential process. Indeed, when several activities could proceed concurrently, the state has to reflect the progress made on each independent activity.

An alternative description of a workflow can be provided by a transition system describing the possible paths from the current state to a goal state. Sometimes, instead of providing a process description, we may specify only the goal state and expect the system to generate a workflow description that could lead to that state through a set of actions. In this case, the new workflow description is generated automatically, knowing a set of tasks and the preconditions and post-conditions for each one of them. In artificial intelligence (AI) this activity is known as *planning*.

The state space of a process includes one initial state and one goal state; a transition system identifies all possible paths from the initial to the goal state. A case corresponds to a particular path in the transition system. The state of a case tracks the progress made during the enactment of that case.

Among the most desirable properties of a process description are the *safety* and *liveness* of the process. Informally, safety means that nothing “bad” ever happens, and liveness means that something “good” will eventually take place should a case based on the process be enacted. Not all processes are safe and live. For example, the process description in Figure 4.2(a) violates the liveness requirement. As long as task C is chosen after completion of B , the process will terminate. However, if D is chosen, then F will never be instantiated, because it requires the completion of both C and E . The process will never terminate, because G requires completion of both D and F .

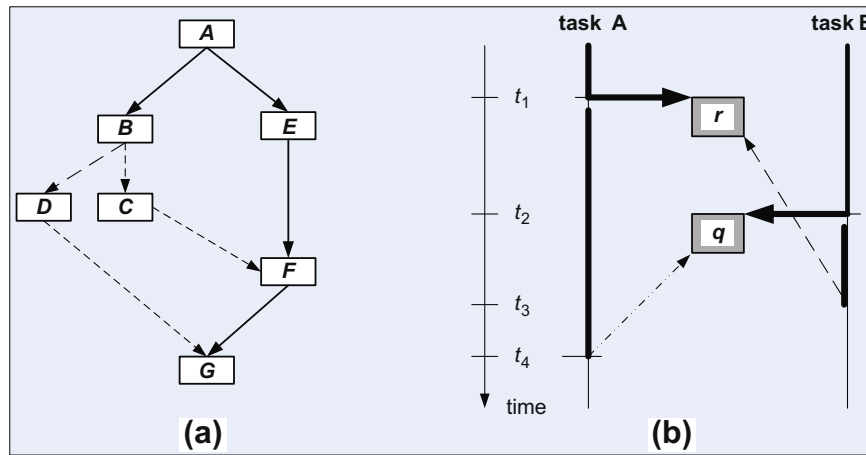
A process description language should be unambiguous and should allow a verification of the process description before the enactment of a case. It is entirely possible that a process description may be enacted correctly in some cases but fail for others. Such enactment failures may be very costly and should be prevented by a thorough verification at the process definition time. To avoid enactment errors, we need to verify process description and check for desirable properties such as safety and liveness. Some process description methods are more suitable for verification than others.

A note of caution: Although the original description of a process could be live, the actual enactment of a case may be affected by deadlocks due to resource allocation. To illustrate this situation, consider two tasks, A and B , running concurrently. Each of them needs exclusive access to resources r and q for a period of time. Either of two scenarios is possible:

1. A or B acquires both resources and then releases them and allows the other task to do the same.
2. We face the undesirable situation in Figure 4.2(b) when, at time t_1 , task A acquires r and continues its execution; then at time t_2 task B acquires q and continues to run. Then at time t_3 task B attempts to acquire r and it blocks because r is under the control of A . Task A continues to run and at time t_4 attempts to acquire q and it blocks because q is under the control of B .

The deadlock illustrated in Figure 4.2(b) can be avoided by requesting each task to acquire all resources at the same time. The price to pay is underutilization of resources. Indeed, the idle time of each resource increases under this scheme.

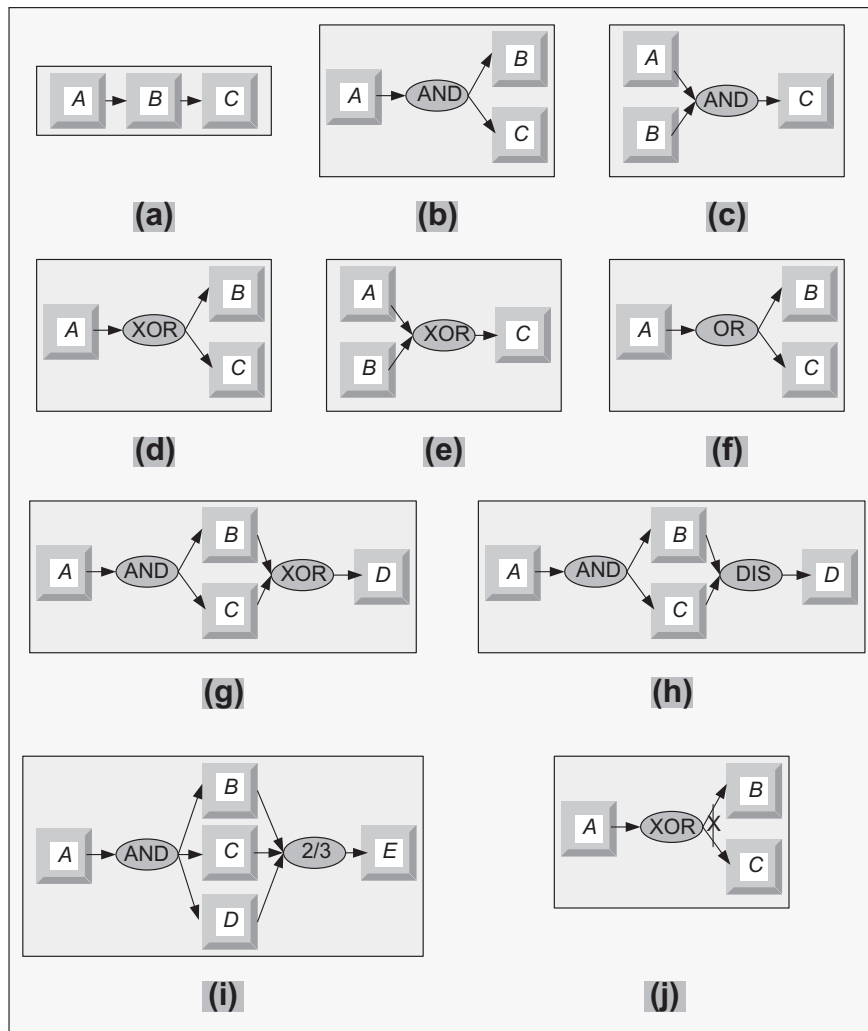
Workflow pattern refers to the temporal relationship among the tasks of a process. The workflow description languages and the mechanisms to control the enactment of a case must have provisions

**FIGURE 4.2**

(a) A process description that violates the liveness requirement. If task C is chosen after completion of B , the process will terminate after executing task G ; if D is chosen, then F will never be instantiated, because it requires the completion of both C and E . The process will never terminate, because G requires completion of both D and F . (b) Tasks A and B need exclusive access to two resources r and q , and a deadlock may take place if the following sequence of events occurs. At time t_1 task A acquires r , at time t_2 task B acquires q and continues to run; then at time t_3 task B attempts to acquire r and it blocks because r is under the control of A . Task A continues to run and at time t_4 attempts to acquire q and it blocks because q is under the control of B .

to support these temporal relationships. Workflow patterns are analyzed in [1,382]. These patterns are classified in several categories: basic, advanced branching and synchronization, structural, state-based, cancellation, and patterns involving multiple instances. The basic workflow patterns illustrated in Figure 4.3 are:

- The *sequence* pattern occurs when several tasks have to be scheduled one after the completion of the other [see Figure 4.3(a)].
- The *AND split* pattern requires several tasks to be executed concurrently. Both tasks B and C are activated when task A terminates [see Figure 4.3(b)]. In case of an *explicit AND split*, the activity graph has a routing node and all activities connected to the routing node are activated as soon as the flow of control reaches the routing node. In the case of an *implicit AND split*, activities are connected directly and conditions can be associated with branches linking an activity with the next ones. Only when the conditions associated with a branch are true are the tasks activated.
- The *synchronization* pattern requires several concurrent activities to terminate before an activity can start. In our example, task C can only start after both tasks A and B terminate [see Figure 4.3(c)].
- The *XOR split* requires a decision; after the completion of task A , either B or C can be activated [see Figure 4.3(d)].

**FIGURE 4.3**

Basic workflow patterns. (a) *Sequence*. (b) *AND split*. (c) *Synchronization*. (d) *XOR split*. (e) *XOR merge*. (f) *OR split*. (g) *Multiple merge*. (h) *Discriminator*. (i) *N out of M join*. (j) *Deferred choice*.

- In the *XOR join*, several alternatives are merged into one. In our example, task C is enabled when either A or B terminates [see Figure 4.3(e)].
- The *OR split* pattern is a construct to choose multiple alternatives out of a set. In our example, after completion of task A, one could activate either B or C, or both [see Figure 4.3(f)].
- The *multiple merge* construct allows multiple activations of a task and does not require synchronization after the execution of concurrent tasks. Once A terminates, tasks B and C execute concurrently

[see Figure 4.3(g)]. When the first of them, say, B , terminates, task D is activated; then when C terminates, D is activated again.

- The *discriminator* pattern waits for a number of incoming branches to complete before activating the subsequent activity [see Figure 4.3(h)]; then it waits for the remaining branches to finish without taking any action until all of them have terminated. Next, it resets itself.
- The *N out of M join* construct provides a barrier synchronization. Assuming that $M > N$ tasks run concurrently, N of them have to reach the barrier before the next task is enabled. In our example, any two out of the three tasks A , B , and C have to finish before E is enabled [see Figure 4.3(i)].
- The *deferred choice* pattern is similar to the XOR split, but this time the choice is not made explicitly and the run-time environment decides what branch to take [see Figure 4.3(j)].

Next we discuss the reachability of the goal state and we consider the following elements:

- A system Σ , an initial state of the system, $\sigma_{initial}$, and a goal state, σ_{goal} .
- A process group $\mathcal{P} = \{p_1, p_2, \dots, p_n\}$; each process p_i in the process group is characterized by a set of preconditions, $pre(p_i)$, post-conditions, $post(p_i)$, and attributes, $atr(p_i)$.
- A workflow described by a directed activity graph $\mathcal{A}$ or by a procedure Π capable of constructing $\mathcal{A}$ given the tuple $\langle \mathcal{P}, \sigma_{initial}, \sigma_{goal} \rangle$. The nodes of $\mathcal{A}$ are processes in $\mathcal{P}$ and the edges define precedence relations among processes. $P_i \rightarrow P_j$ implies that $pre(p_j) \subset post(p_i)$.
- A set of constraints $\mathcal{C} = \{C_1, C_2, \dots, C_m\}$.

The coordination problem for system Σ in state $\sigma_{initial}$ is to reach state σ_{goal} as a result of post-conditions of some process $P_{final} \in \mathcal{P}$ subject to constraints $C_i \in \mathcal{C}$. Here $\sigma_{initial}$ enables the preconditions of some process $P_{initial} \in \mathcal{P}$. Informally, this means that a chain of processes exists such that the post-conditions of one process are preconditions of the next process in the chain.

Generally, the preconditions of a process are either the conditions and/or the events that trigger the execution of the process or the data the process expects as input; the post-conditions are the results produced by the process. The attributes of a process describe special requirements or properties of the process.

Some workflows are static. The activity graph does not change during the enactment of a case. *Dynamic workflows* are those that allow the activity graph to be modified during the enactment of a case. Some of the more difficult questions encountered in dynamic workflow management refer to (i) how to integrate workflow and resource management and guarantee optimality or near optimality of cost functions for individual cases; (ii) how to guarantee consistency after a change in a workflow; and (iii) how to create a dynamic workflow. Static workflows can be described in WFDL (the workflow definition language), but dynamic workflows need a more flexible approach.

We distinguish two basic models for the mechanics of workflow enactment:

1. *Strong coordination models*, whereby the process group $\mathcal{P}$ executes under the supervision of a *coordinator* process or processes. A coordinator process acts as an enactment engine and ensures a seamless transition from one process to another in the activity graph.
2. *Weak coordination models*, whereby there is no supervisory process.

In the first case, we may deploy a *hierarchical coordination scheme* with several levels of coordinators. A supervisor at level i in a hierarchical scheme with $i + 1$ levels coordinates a subset of processes in the process group. A supervisor at level $i - 1$ coordinates a number of supervisors at level i and the root provides global coordination. Such a hierarchical coordination scheme may be used to reduce the communication overhead; a coordinator and the processes it supervises may be colocated.

The most important feature of this coordination model is the ability to support dynamic workflows. The coordinator or the global coordinator may respond to a request to modify the workflow by first stopping all the threads of control in a consistent state, then investigating the feasibility of the requested changes, and finally, implementing feasible changes.

Weak coordination models are based on peer-to-peer communication between processes in the process group by means of a societal service such as a *tuple space*. Once a process $p_i \in \mathcal{P}$ finishes, it deposits a token, including possibly a subset of its post-conditions, $post(p_i)$, in a tuple space. The consumer process p_j is expected to visit the tuple space at some point in time, examine the tokens left by its ancestors in the activity graph, and, if its preconditions $pre(p_j)$ are satisfied, commence the execution. This approach requires individual processes to either have a copy of the activity graph or some timetable to visit the tuple space. An alternative approach is using an *active space*, a tuple space augmented with the ability to generate an event awakening the consumer of a token.

There are similarities and some differences between workflows of traditional transaction-oriented systems and *cloud workflows*. The similarities are mostly at the modeling level, whereas the differences affect the mechanisms used to implement workflow management systems. Some of the more subtle differences between the two are:

- The emphasis in a transactional model is placed on the contractual aspect of a transaction; in a workflow the enactment of a case is sometimes based on a “best-effort” model whereby the agents involved will do their best to attain the goal state but there is no guarantee of success.
- A critical aspect of the transactional model in database applications is maintaining a consistent state of the database; however, a cloud is an open system, and thus its state is considerably more difficult to define.
- The database transactions are typically short-lived; the tasks of a cloud workflow could be long-lasting.
- A database transaction consists of a set of well-defined actions that are unlikely to be altered during the execution of the transaction. However, the process description of a cloud workflow may change during the lifetime of a case.
- The individual tasks of a cloud workflow may not exhibit the traditional properties of database transactions. For example, consider durability: At any instance of time, before reaching the goal state, a workflow may roll back to some previously encountered state and continue from there on an entirely different path. A task of a workflow could be either reversible or irreversible. Sometimes, paying a penalty for reversing an action is more profitable in the long run than continuing on a wrong path.
- Resource allocation is a critical aspect of the workflow enactment on a cloud without an immediate correspondent for database transactions.

The relatively simple coordination model discussed next is often used in cloud computing.

4.5 Coordination based on a state machine model: The *ZooKeeper*

Cloud computing elasticity requires the ability to distribute computations and data across multiple systems. Coordination among these systems is one of the critical functions to be exercised in a distributed environment. The coordination model depends on the specific task, such as coordination of data storage, orchestration of multiple activities, blocking an activity until an event occurs, reaching consensus for the next action, or recovery after an error.

The entities to be coordinated could be processes running on a set of cloud servers or even running on multiple clouds. Servers running critical tasks are often replicated, so when one primary server fails, a backup automatically continues the execution. This is only possible if the backup is in a *hot standby* mode – in other words, the standby server shares the same state at all times with the primary.

For example, in the distributed data store model discussed in Section 3.5, the access to data is mitigated by a proxy. This proxy is a single point of failure; thus, an architecture with multiple proxies is desirable. These proxies should be in the same state so that, whenever one of them fails, the client could seamlessly continue to access the data using another proxy.

Consider now an advertising service that involves a large number of servers in a cloud. The advertising service runs on a number of servers specialized for tasks such as database access, monitoring, accounting, event logging, installers, customer dashboards,⁵ advertising campaign planners, scenario testing, and so on. A solution to coordinate these activities is through configuration files shared by all systems. When the service starts or after a system failure, all servers use the configuration file to coordinate their actions. This solution is static. Any change requires an update and redistribution of the configuration file. Moreover, in case of a system failure the configuration file does not allow recovery from the state of each server prior to the system crash, which is a more desirable alternative.

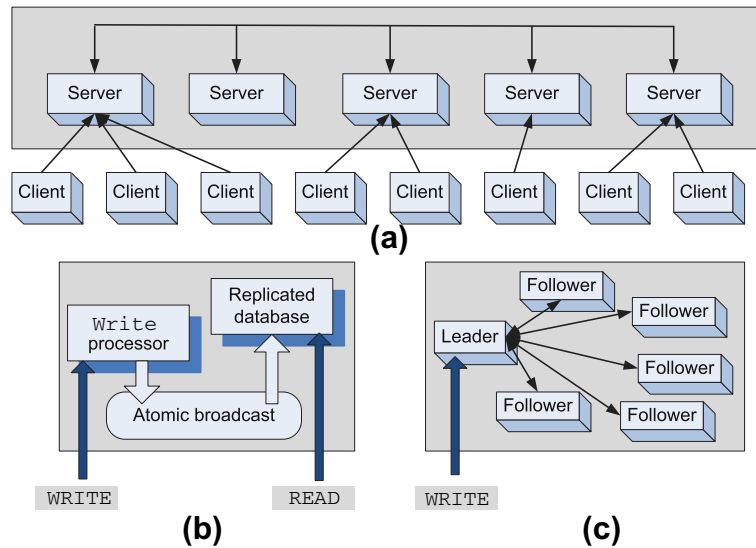
A solution for the proxy coordination problem is to consider a proxy as a deterministic finite state machine that performs the commands sent by clients in some sequence. The proxy has thus a definite state and, when a command is received, it transitions to another state. When P proxies are involved, all of them must be synchronized and must execute the same sequence of state machine commands; this can be ensured if all proxies implement a version of the Paxos consensus algorithm described in Section 2.11.

ZooKeeper is a distributed coordination service based on this model. The high-throughput and low-latency service is used for coordination in large-scale distributed systems. The open-source software is written in Java and has bindings for Java and C. Information about the project is available at <http://zookeeper.apache.org/>.

The *ZooKeeper* software must first be downloaded and installed on several servers; then clients can connect to any one of these servers and access the coordination service. The service is available as long as the majority of servers in the pack are available.

The organization of the service is shown in Figure 4.4. The servers in the pack communicate with one another and elect a *leader*. A database is replicated on each one of them and the consistency of the

⁵A customer dashboard provides access to key customer information, such as contact name and account number, in an area of the screen that remains persistent as the user navigates through multiple Web pages.

**FIGURE 4.4**

The *ZooKeeper* coordination service. (a) The service provides a single system image. Clients can connect to any server in the pack. (b) Functional model of the *ZooKeeper* service. The replicated database is accessed directly by `read` commands; `write` commands involve more intricate processing based on atomic broadcast. (c) Processing a `write` command: (1) A server receiving the command from a client forwards the command to the *leader*; (2) the *leader* uses atomic broadcast to reach consensus among all *followers*.

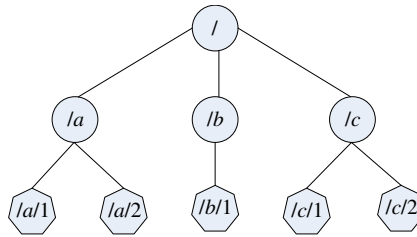
replicas is maintained. Figure 4.4(a) shows that the service provides a single system image. A client can connect to any server of the pack.

A client uses TCP to connect to a single server. Through the TCP connection a client sends requests and receives responses and watches events. A client synchronizes its clock with the server. If the server fails, the TCP connections of all clients connected to it time out and the clients detect the failure of the server and connect to other servers.

Figures 4.4(b) and (c) show that a `read` operation directed to any server in the pack returns the same result, whereas the processing of a `write` operation is more involved; the servers elect a *leader*, and any *follower* receiving a request from one of the clients connected to it forwards it to the leader. The leader uses atomic broadcast to reach consensus. When the leader fails, the servers elect a new leader.

The system is organized as a shared hierarchical namespace similar to the organization of a file system. A name is a sequence of path elements separated by a backslash. Every name in *ZooKeeper*'s namespace is identified by a unique path (see Figure 4.5).

In *ZooKeeper* the *znodes*, the equivalent of the *inodes* of a file system, can have data associated with them. Indeed, the system is designed to store state information. The data in each node includes version

**FIGURE 4.5**

ZooKeeper is organized as a shared hierarchical namespace in which a name is a sequence of path elements separated by a backslash.

numbers for the data, changes of ACLs,⁶ and time stamps. A client can set a watch on a *znode* and receive a notification when the *znode* changes. This organization allows coordinated updates. The data retrieved by a client also contains a version number. Each update is stamped with a number that reflects the order of the transition.

The data stored in each node is read and written atomically. A `read` returns all the data stored in a *znode*, whereas a `write` replaces all the data in the *znode*. Unlike in a file system, *ZooKeeper* data, the image of the state, is stored in the server memory. Updates are logged to disk for recoverability, and *writes* are serialized to disk before they are applied to the in-memory database that contains the entire tree. The *ZooKeeper* service guarantees:

1. *Atomicity*. A transaction either completes or fails.
2. *Sequential consistency of updates*. Updates are applied strictly in the order in which they are received.
3. *Single system image for the clients*. A client receives the same response regardless of the server it connects to.
4. *Persistence of updates*. Once applied, an update persists until it is overwritten by a client.
5. *Reliability*. The system is guaranteed to function correctly as long as the majority of servers function correctly.

To reduce the response time, `read` requests are serviced from the local replica of the server that is connected to the client. When the leader receives a `write` request, it determines the state of the system where the `write` will be applied and then it transforms the state into a transaction that captures this new state.

The messaging layer is responsible for the election of a new leader when the current leader fails. The messaging protocol uses *packets* (sequences of bytes sent through a FIFO channel), *proposals* (units of agreement), and *messages* (sequences of bytes atomically broadcast to all servers). A message is included in a proposal and it is agreed on before it is delivered. Proposals are agreed on by exchanging packets with a quorum of servers, as required by the Paxos algorithm.

⁶An access control list (ACL) is a list of pairs (subject,value) that defines the list of access rights to an object; for example, read, write, and execute permissions for a file.

An atomic messaging system keeps all the servers in a pack in synch. This system guarantees (a) reliable delivery: if message m is delivered to one server, it will be eventually delivered to all servers; (b) total order: if message m is delivered before message n to one server, m will be delivered before n to all servers; and (c) causal order: if message n is sent after m has been delivered by the sender of n , then m must be ordered before n .

The application programming interface (API) to the *ZooKeeper* service is very simple and consists of seven operations:

- *create* – add a node at a given location on the tree.
- *delete* – delete a node.
- *get data* – read data from a node.
- *set data* – write data to a node.
- *get children* – retrieve a list of the children of the node.
- *synch* – wait for the data to propagate.

The system also supports the creation of *ephemeral* nodes, which are nodes that are created when a session starts and deleted when the session ends.

This brief description shows that the *ZooKeeper* service supports the finite state machine model of coordination. In this case a *znode* stores the state. The *ZooKeeper* service can be used to implement higher-level operations such as group membership, synchronization, and so on. The system is used by Yahoo!’s Message Broker and by several other applications.

4.6 The *MapReduce* programming model

A main advantage of cloud computing is elasticity – the ability to use as many servers as necessary to optimally respond to the cost and the timing constraints of an application. In the case of transaction processing systems, typically a front-end system distributes the incoming transactions to a number of back-end systems and attempts to balance the load among them. As the workload increases, new back-end systems are added to the pool.

For data-intensive batch applications, partitioning the workload is not always trivial. Only in some cases can the data be partitioned into blocks of arbitrary size and processed in parallel by servers in the cloud. We distinguish two types of divisible workloads:

- *Modularly divisible*. The workload partitioning is defined a priori.
- *Arbitrarily divisible*. The workload can be partitioned into an arbitrarily large number of smaller workloads of equal or very close size.

Many realistic applications in physics, biology, and other areas of computational science and engineering obey the arbitrarily divisible load-sharing model. The Divisible Load Theory (DLT) is analyzed in the literature (see Section 4.12).

MapReduce is based on a very simple idea for parallel processing of data-intensive applications supporting arbitrarily divisible load sharing. First, split the data into blocks, assign each block to an instance or process, and run these instances in parallel. Once all the instances have finished, the computations assigned to them start the second phase: Merge the partial results produced by individual

instances. The so-called same program, multiple data (SPMD) paradigm, used since the early days of parallel computing, is based on the same idea but assumes that a *master* instance partitions the data and gathers the partial results.

MapReduce is a programming model inspired by the *Map* and the *Reduce* primitives of the LISP programming language. It was conceived for processing and generating large data sets on computing clusters [100]. As a result of the computation, a set of input $\langle \text{key}, \text{value} \rangle$ pairs is transformed into a set of output $\langle \text{key}, \text{value} \rangle$ pairs.

Numerous applications can be easily implemented using this model. For example, one can process logs of Web page requests and count the URL access frequency. The *Map* function outputs the pairs $\langle \text{URL}, 1 \rangle$ and the *Reduce* function produces the pairs $\langle \text{URL}, \text{totalcount} \rangle$. Another trivial example is *distributed sort* when the map function extracts the key from each record and produces a $\langle \text{key}, \text{record} \rangle$ pair and the *Reduce* function outputs these pairs unchanged. The following example [100] shows the two user-defined functions for an application that counts the number of occurrences of each word in a set of documents.

```
map(String key, String value):
    //key: document name; value: document contents
    for each word w in value:
        EmitIntermediate (w, "1");

reduce (String key, Iterator values):
    // key: a word; values: a list of counts
    int result = 0;
    for each v in values:
        result += ParseInt (v);
    Emit (AsString (result));
```

Call M and R the number of *Map* and *Reduce* tasks, respectively, and N the number of systems used by the *MapReduce*. When a user program invokes the *MapReduce function*, the following sequence of actions take place (see Figure 4.6):

1. The run-time library splits the input files into M splits of 16 to 64 MB each, identifies a number N of systems to run, and starts multiple copies of the program, one of the system being a *master* and the others *workers*. The *master* assigns to each idle system either a *Map* or a *Reduce* task. The *master* makes $\mathcal{O}(M + R)$ scheduling decisions and keeps $\mathcal{O}(M \times R)$ worker state vectors in memory. These considerations limit the size of M and R ; at the same time, efficiency considerations require that $M, R \gg N$.
2. A worker being assigned a *Map* task reads the corresponding input split, parses $\langle \text{key}, \text{value} \rangle$ pairs, and passes each pair to a user-defined *Map* function. The intermediate $\langle \text{key}, \text{value} \rangle$ pairs produced by the *Map* function are buffered in memory before being written to a local disk and partitioned into R regions by the partitioning function.
3. The locations of these buffered pairs on the local disk are passed back to the *master*, who is responsible for forwarding these locations to the *Reduce* workers. A *Reduce* worker uses remote procedure calls to read the buffered data from the local disks of the *Map* workers; after reading all the

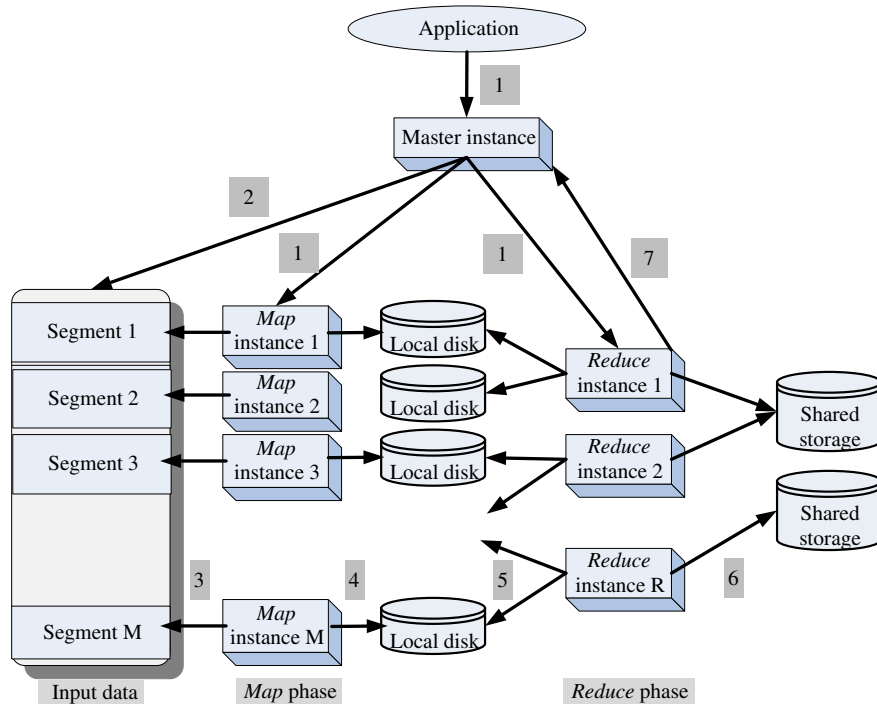


FIGURE 4.6

The *MapReduce* philosophy. (1) An application starts a *master instance* and M worker instances for the *Map* phase and, later, R worker instances for the *Reduce* phase. (2) The *master* partitions the input data in M segments. (3) Each *Map instance* reads its input data segment and processes the data. (4) The results of the processing are stored on the local disks of the servers where the *Map instances* run. (5) When all *Map instances* have finished processing their data, the R *Reduce instances* read the results of the first phase and merge the partial results. (6) The final results are written by the *Reduce instances* to a shared storage server. (7) The *master instance* monitors the *Reduce instances* and, when all of them report task completion, the application is terminated.

intermediate data, it sorts it by the intermediate keys. For each unique intermediate key, the key and the corresponding set of intermediate values are passed to a user-defined *Reduce* function. The output of the *Reduce* function is appended to a final output file.

4. When all *Map* and *Reduce* tasks have been completed, the *master* wakes up the user program.

The system is fault tolerant. For each *Map* and *Reduce* task, the *master* stores the state (idle, in-progress, or completed) and the identity of the worker machine. The *master* pings every worker periodically and marks the worker as failed if it does not respond. A task in progress on a failed worker is reset to idle and becomes eligible for rescheduling. The *master* writes periodic checkpoints of its control data

structures and, if the task fails, it can be restarted from the last checkpoint. The data is stored using GFS, the Google File System, discussed in Section 8.5.

An environment for experimenting with *MapReduce* is described in [100]: The computers are typically dual-processor *x86* running *Linux*, with 2–4 GB of memory per machine and commodity networking hardware typically 100–1,000 Mbps. A cluster consists of hundreds or thousands of machines. Data is stored on IDE⁷ disks attached directly to individual machines. The file system uses replication to provide availability and reliability with unreliable hardware. To minimize network bandwidth, the input data is stored on the local disks of each system.

4.7 A case study: The *GrepTheWeb* application

An application called *GrepTheWeb*, discussed in [360], is now in production at Amazon. We use it to illustrate the power and appeal of cloud computing. The application allows a user to define a regular expression and search the Web for records that match it. *GrepTheWeb* is analogous to the *Unix grep* command used to search a file for a given regular expression.

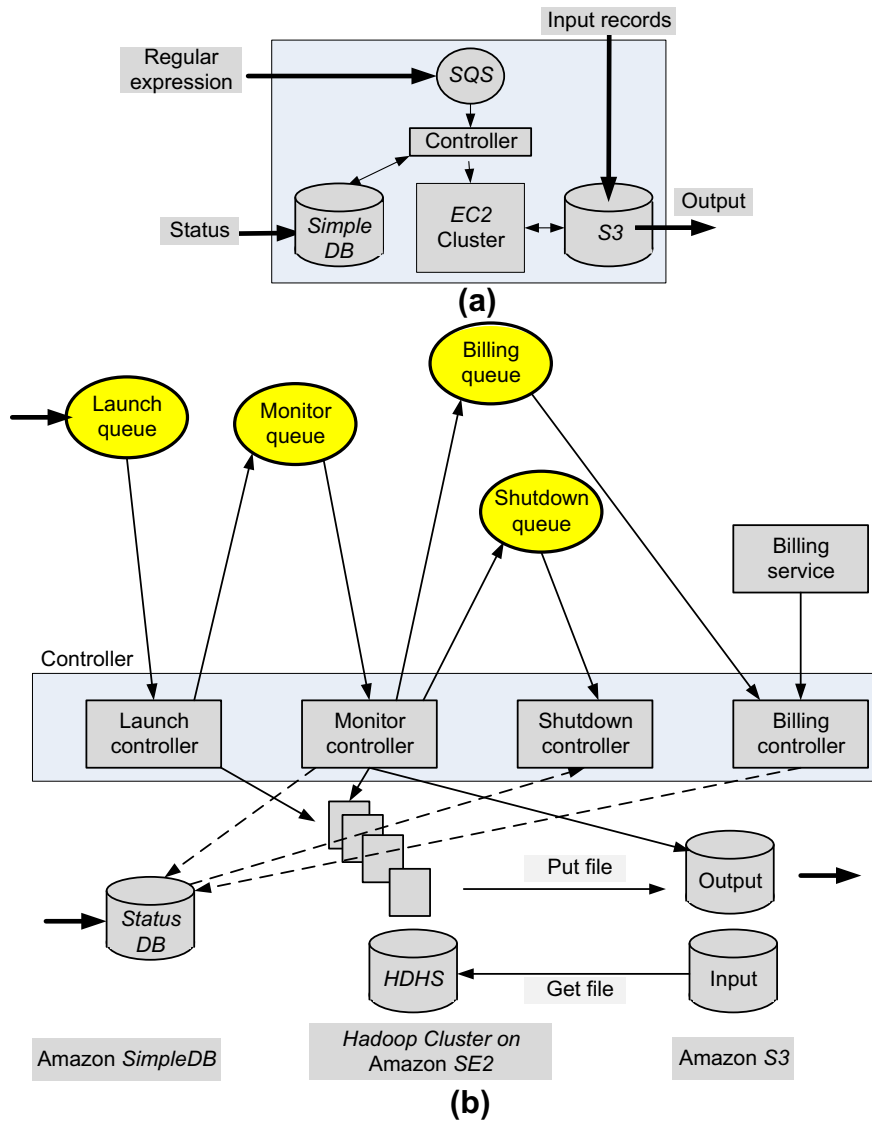
This application performs a search of a very large set of records, attempting to identify records that satisfy a regular expression. The source of this search is a collection of document URLs produced by the *Alexa Web Search*, a software system that crawls the Web every night. The inputs to the applications are a regular expression and the large data set produced by the Web-crawling software; the output is the set of records that satisfy the expression. The user is able to interact with the application and get the current status [see Figure 4.7(a)].

The application uses message passing to trigger the activities of multiple controller threads that launch the application, initiate processing, shut down the system, and create billing records. *GrepTheWeb* uses *Hadoop MapReduce*, an open-source software package that splits a large data set into chunks, distributes them across multiple systems, launches the processing, and, when the processing is complete, aggregates the outputs from different systems into a final result. *Apache Hadoop* is a software library for distributed processing of large data sets across clusters of computers using a simple programming model.

The details of the workflow of *GrepTheWeb* are captured in Figure 4.7(b) and consist of the following steps [360]:

1. *The startup phase.* Creates several queues – launch, monitor, billing, and shutdown queues. Starts the corresponding controller threads. Each thread periodically polls its input queue and, when a message is available, retrieves the message, parses it, and takes the required actions.
2. *The processing phase.* This phase is triggered by a `StartGrep` user request; then a launch message is enqueued in the launch queue. The launch controller thread picks up the message and executes the launch task; then, it updates the status and time stamps in the Amazon *Simple DB* domain. Finally, it enqueues a message in the monitor queue and deletes the message from the launch queue. The processing phase consists of the following steps:

⁷Integrated Drive Electronics (IDE) is an interface for connecting disk drives. The drive controller is integrated into the drive, as opposed to a separate controller on or connected to the motherboard.

**FIGURE 4.7**

The organization of the *GrepTheWeb* application. The application uses the *Hadoop MapReduce* software and four Amazon services: *EC2*, *Simple DB*, *S3*, and *SQS*. (a) The simplified workflow showing the two inputs, the regular expression and the input records generated by the Web crawler. A third type of input is the user commands to report the current status and to terminate the processing. (b) The detailed workflow; the system is based on message passing between several queues; four controller threads periodically poll their associated input queues, retrieve messages, and carry out the required actions.

- a. The launch task starts Amazon *EC2* instances. It uses a Java Runtime Environment preinstalled Amazon Machine Image (AMI), deploys required *Hadoop* libraries, and starts a *Hadoop* Job (run *Map/Reduce* tasks).
 - b. *Hadoop* runs map tasks on Amazon *EC2* slave nodes in parallel. A map task takes files from Amazon *S3*, runs a regular expression, and writes the match results locally, along with a description of up to five matches. Then the combine/reduce task combines and sorts the results and consolidates the output.
 - c. Final results are stored on Amazon *S3* in the output bucket.
3. *The monitoring phase.* The monitor controller thread retrieves the message left at the beginning of the processing phase, validates the status/error in Amazon *Simple DB*, and executes the monitor task. It updates the status in the Amazon *Simple DB* domain and enqueues messages in the shutdown and billing queues. The monitor task checks for the *Hadoop* status periodically and updates the *Simple DB* items with status/error and the Amazon *S3* output file. Finally, it deletes the message from the monitor queue when the processing is completed.
 4. *The shutdown phase.* The shutdown controller thread retrieves the message from the shutdown queue and executes the shutdown task, which updates the status and time stamps in the Amazon *Simple DB* domain. Finally, it deletes the message from the shutdown queue after processing. The shutdown phase consists of the following steps:
 - a. The shutdown task kills the *Hadoop* processes, terminates the *EC2* instances after getting *EC2* topology information from Amazon *Simple DB*, and disposes of the infrastructure.
 - b. The billing task gets the *EC2* topology information, *Simple DB* usage, and *S3* file and query input, calculates the charges, and passes the information to the billing service.
 5. *The cleanup phase.* Archives the *Simple DB* data with user info.
 6. *User interactions with the system.* Get the status and output results. The *GetStatus* is applied to the service endpoint to get the status of the overall system (all controllers and *Hadoop*) and download the filtered results from Amazon *S3* after completion.

To optimize the end-to-end transfer rates in the *S3* storage system, multiple files are bundled up and stored as *S3* objects. Another performance optimization is to run a script and sort the keys and the URL pointers and upload them in sorted order to *S3*. In addition, multiple fetch threads are started in order to fetch the objects.

This application illustrates the means to create an on-demand infrastructure and run it on a massively distributed system in a manner that allows it to run in parallel and scale up and down based on the number of users and the problem size.

4.8 Clouds for science and engineering

For more than two thousand years of human history, science was empirical. Several hundred years ago theoretical methods based on models and generalization were introduced, allowing substantial progress in human knowledge. In the last few decades, we have witnessed the explosion of computational science based on the simulation of complex phenomena.